KeenPay

Production Database Schema

Simple, secure PostgreSQL design for agentic commerce.
AI proposes. The control plane authorizes. Money never bypasses policy.

Grow | Sell | Protect

PostgreSQL 15 | Integer Paise | Append-Only Audit

Version 1.0.0 | August 2026

1. Database Overview

KeenPay uses PostgreSQL as the source of truth for catalog, negotiation sessions, orders, and audit. v1 targets a single merchant pilot with merchant_id columns ready for multi-tenant expansion. The AI runtime reads catalog and writes session proposals; financial tables are owned by the deterministic control plane.

Table Groups

Group	Tables
Catalog	products
Agentic Checkout	negotiation_sessions, langgraph_checkpoints
Commerce	orders, inventory_holds
Control & Safety	escalation_tickets
Payments	webhook_events
Audit	audit_logs (append-only)

Design Principles

- Integer paise (BIGINT/INTEGER) for all money - no floats
- AI may propose offers in negotiation_sessions; only policy-approved amounts reach orders
- audit_logs is append-only; UPDATE/DELETE blocked by database trigger
- Every payment link binds to guardrail_decision_id + offer_version
- Webhook events deduplicated by unique event_id

2. Data Plane Architecture

The schema enforces a strict split: LangGraph mirrors conversational state in negotiation_sessions and langgraph_checkpoints. Once guardrails approve and the user confirms, an immutable order row is created. Payment lifecycle completes via Razorpay webhooks stored in webhook_events.

UNTRUSTED INPUT	User chat, LLM proposals, inbound webhooks
LANGGRAPH STATE	negotiation_sessions, langgraph_checkpoints
CONTROL PLANE	Policy binding via guardrail_decision_id on orders
DURABLE AUDIT	audit_logs, webhook_events, escalation_tickets

Entity Relationships (v1)

products <- negotiation_sessions (via selected_line_items JSONB) -> orders -> webhook_events. audit_logs correlates session_id, order_id, and decision_id across the lifecycle. inventory_holds tie stock reservations to session_id before payment link creation.

3. Catalog Tables

TABLE: products

Purpose

Merchant catalog. Price and cost basis are read by the policy engine for margin guardrails. The AI runtime may read active products but cannot mutate prices.

Column	Type	Meaning
id	VARCHAR(64)	Product ID
sku	VARCHAR(64)	Stock keeping unit (unique per merchant)
merchant_id	VARCHAR(64)	Merchant scope (default merchant_keen)
name	VARCHAR(255)	Display name
list_price_paise	INTEGER	List price in paise
cost_paise	INTEGER	Cost basis for RULE_MIN_MARGIN
quantity_on_hand	INTEGER	Physical stock
quantity_reserved	INTEGER	Reserved by active holds
attributes	JSONB	Category, color, size, etc.
search_vector	TSVECTOR	Full-text search index (generated)
active	BOOLEAN	Catalog visibility

Foreign Keys

- tenant_id -> merchants (v1 implicit single merchant)

Important Indexes

- (merchant_id, sku) UNIQUE
- GIN on search_vector
- GIN on attributes

Service Access

- AI Runtime (LangGraph): yes
- Policy Engine: yes
- Payment Service: no direct access
- Webhook Worker: no direct access

4. Agentic Checkout Tables

TABLE: negotiation_sessions

Purpose

Live checkout session. Mirrors LangGraph KeenPayState: intent, offers, guardrail binding, and user confirmation gate. No payment link without APPROVED guardrail + user_confirmed_payment.

Column	Type	Meaning
id	UUID	Session ID (= LangGraph thread_id)
status	ENUM	active | negotiating | awaiting_confirmation | payment_pending | paid | escalated | closed
negotiation_round	INTEGER	Bounded negotiation counter (max 5)
offer_version	INTEGER	Monotonic offer version
parsed_intent	JSONB	Structured intent from parse_intent node
proposed_offer	JSONB	LLM-proposed offer (untrusted until guardrail)
approved_offer	JSONB	Policy-approved offer snapshot
guardrail_decision	ENUM	APPROVED | REJECTED | ESCALATED
guardrail_decision_id	UUID	Links to audit_logs decision
guardrail_detail	JSONB	Per-rule evaluation results
user_confirmed_payment	BOOLEAN	Explicit user consent gate
final_amount_paise	INTEGER	Deterministic total (post compute_totals)
security_block	BOOLEAN	True after prompt injection / anomaly block
langgraph_thread_id	UUID	LangGraph checkpointer reference

Foreign Keys

- langgraph_thread_id conventionally equals id

Important Indexes

- (user_id, created_at DESC)
- (status) partial index
- (guardrail_decision_id)

Service Access

- AI Runtime (LangGraph): yes
- Policy Engine: yes
- Payment Service: no direct access
- Merchant Dashboard: yes

TABLE: langgraph_checkpoints

Purpose

LangGraph durable state snapshots for conversation recovery and interrupt/resume.

Column	Type	Meaning
thread_id	UUID	Session thread
checkpoint_id	UUID	Checkpoint version
checkpoint	JSONB	Serialized graph state
metadata	JSONB	Node hints, timestamps

Foreign Keys

- thread_id -> negotiation_sessions.langgraph_thread_id

Important Indexes

- (thread_id, created_at DESC)

Service Access

- AI Runtime (LangGraph): yes
- Policy Engine: no direct access
- Payment Service: no direct access

5. Commerce & Inventory Tables

TABLE: orders

Purpose

Frozen purchase created only after guardrail APPROVED + user confirmation. Amounts and line_items are immutable after insert. Binds to Razorpay payment link.

Column	Type	Meaning
id	VARCHAR(64)	Order ID
session_id	UUID	Source negotiation session
status	ENUM	pending | paid | expired | cancelled | payment_disputed
subtotal_paise	INTEGER	Pre-discount subtotal
discount_amount_paise	INTEGER	Applied discount
final_amount_paise	INTEGER	Charge amount (must match webhook)
line_items	JSONB	Immutable line snapshot at order time
guardrail_decision_id	UUID	Required policy evaluation reference
offer_version	INTEGER	Offer version that created this order
policy_version	VARCHAR(32)	Policy ruleset version
razorpay_payment_link_id	VARCHAR(64)	Razorpay entity
razorpay_payment_link_url	TEXT	Checkout URL for buyer
idempotency_key	VARCHAR(128)	Duplicate protection (UNIQUE)

Foreign Keys

- session_id -> negotiation_sessions.id
- guardrail_decision_id -> audit trail (logical)

Important Indexes

- (session_id)
- (razorpay_payment_link_id) partial
- UNIQUE (idempotency_key)
- UNIQUE pending order per session

Service Access

- AI Runtime: no direct access
- Payment Service: yes
- Webhook Worker: yes
- Reconciliation Worker: yes

TABLE: inventory_holds

Purpose

Durable stock reservation during checkout. Complements Redis soft holds. Released on expiry/cancel; consumed on payment capture.

Column	Type	Meaning
id	UUID	Hold ID
session_id	UUID	Checkout session
product_id	VARCHAR(64)	Product reserved
sku	VARCHAR(64)	SKU for guardrail reads
quantity	INTEGER	Reserved quantity
expires_at	TIMESTAMPTZ	Auto-release time (15 min default)
released_at	TIMESTAMPTZ	Null while active

Foreign Keys

- session_id -> negotiation_sessions.id
- product_id -> products.id

Important Indexes

- (expires_at) WHERE released_at IS NULL
- UNIQUE (session_id, sku)

Service Access

- AI Runtime: no direct access
- Policy Engine: yes
- Payment Service: yes

6. Control, Payments & Audit Tables

TABLE: escalation_tickets

Purpose

Human-in-the-loop queue when guardrails ESCALATE (anomaly, max rounds, engine error). Human overrides are logged with actor=human; margin floor is never overridden in v1.

Column	Type	Meaning
id	UUID	Ticket ID
session_id	UUID	Blocked session
priority	VARCHAR(4)	P0 | P1 | P2
reason_code	VARCHAR(64)	e.g. RULE_SECURITY_ANOMALY
status	VARCHAR(16)	open | assigned | resolved | expired
proposed_offer_snapshot	JSONB	Offer under review
policy_snapshot	JSONB	Policy at escalation time
resolution	VARCHAR(32)	approve_override | deny | counter_offer
override_discount_pct	NUMERIC(5,2)	Manager override (bounded)

Foreign Keys

- session_id -> negotiation_sessions.id

Important Indexes

- (status, priority, created_at)

Service Access

- AI Runtime: no direct access
- Control Plane / Admin API: yes
- Merchant Dashboard: yes

TABLE: webhook_events

Purpose

Razorpay inbound events. Signature verified on receipt; deduplicated by event_id. Amount mismatch marks order payment_disputed - never auto-paid.

Column	Type	Meaning
id	UUID	Internal event ID
event_id	VARCHAR(128)	Razorpay event ID (UNIQUE)
event_type	VARCHAR(64)	e.g. payment_link.paid
payload	JSONB	Raw webhook body
signature_valid	BOOLEAN	HMAC verification result
processed	BOOLEAN	Worker completion flag
order_id	VARCHAR(64)	Matched order

Foreign Keys

- order_id -> orders.id

Important Indexes

- UNIQUE (event_id)
- (event_type, received_at DESC)
- unprocessed index

Service Access

- AI Runtime: no direct access
- Webhook Worker: yes
- Payment Service: no direct access

TABLE: audit_logs

Purpose

Append-only tamper-evident ledger. Every money action and guardrail evaluation writes a row with input_snapshot and output_snapshot. UPDATE/DELETE blocked by trigger.

Column	Type	Meaning
id	UUID	Audit row ID
session_id	UUID	Negotiation session
order_id	VARCHAR(64)	Order if applicable
actor	ENUM	agent | policy_engine | user | system | webhook | human
action	VARCHAR(128)	e.g. GUARDRAIL_EVALUATED, PAYMENT_LINK_CREATED
decision_id	UUID	Guardrail evaluation reference
offer_version	INTEGER	Bound offer version
input_snapshot	JSONB	Proposed offer, policy version, message hash
output_snapshot	JSONB	Decision, rule results, payment link ID
trace_metadata	JSONB	Node name, duration_ms, anomaly score

Foreign Keys

- session_id -> negotiation_sessions.id
- order_id -> orders.id

Important Indexes

- (session_id, created_at DESC)
- (decision_id) partial
- GIN on trace_metadata

Service Access

- AI Runtime: no direct access
- Policy Engine: yes
- Merchant Dashboard (read): yes
- Webhook Worker: yes

7. State Transitions

negotiation_sessions.status

From	To / Action
active	negotiating
negotiating	awaiting_confirmation (guardrail APPROVED)
awaiting_confirmation	payment_pending (user confirmed)
payment_pending	paid (webhook verified)
*	escalated (guardrail ESCALATED)
*	closed (terminal)

orders.status

From	To / Action
pending	paid (webhook amount match)
pending	expired (link TTL)
pending	cancelled
pending	payment_disputed (amount mismatch)

guardrail_decision

From	To / Action
proposed offer	APPROVED -> compute_totals
proposed offer	REJECTED -> explain + retry (max 5 rounds)
proposed offer	ESCALATED -> escalation_tickets

SELL Database Flow

- 1. User message -> negotiation_sessions updated (proposed_offer)
- 2. Policy engine -> guardrail_decision + audit_logs row
- 3. User confirms -> inventory_holds + orders (pending) + Razorpay link
- 4. Webhook -> webhook_events + orders.status=paid + audit_logs

8. Transaction Passport (Derived View)

There is no separate passport table. The Transaction Passport is assembled from negotiation_sessions, orders, audit_logs, and webhook_events for support replay and compliance. The frontend trace panel streams real-time events via Redis; audit_logs is the durable source of truth.

Source of Truth

Thing	Where it lives
Product price	products.list_price_paise
Negotiated unit price	approved_offer in negotiation_sessions
Final charge amount	orders.final_amount_paise
Policy evaluation	audit_logs where action=GUARDRAIL_EVALUATED
Payment status	orders.status + webhook_events
Provider truth	Razorpay API + verified webhook
Hot session cache	Redis (never authoritative for money)

9. Production Checklist

- All money stored as integer paise - no floating point
- Payment link requires guardrail_decision=APPROVED + user_confirmed_payment
- Idempotency key on every order and cached Razorpay link per offer version
- Webhook HMAC verified; duplicate event_id returns 200 without side effect
- Webhook amount must equal orders.final_amount_paise or mark payment_disputed
- audit_logs append-only (trigger blocks UPDATE/DELETE)
- Inventory: quantity_reserved <= quantity_on_hand constraint
- LLM never writes to orders or webhook_events - gated Python nodes only
- Negotiation capped at 5 rounds before ESCALATED -> escalation_tickets
- Secrets in environment / secrets manager - never in database or LLM context

10. v1.1 Roadmap (Not in Current DDL)

The following enterprise tables are deferred to keep v1 deployable: separate tenants/users with RLS, standalone policies/authorizations tables, refunds, idempotency_keys cache table, transactional outbox, and GROW campaign tables. v1 encodes policy in MerchantPolicy config and guardrail snapshots in audit_logs JSONB.